

Object-Oriented Programming

Unit 2: Features of OOP — Encapsulation, Object Identity & Polymorphism

MAKAUT B.Tech IT | Exam-Ready Notes with Java Code

Note on this unit: Your MAKAUT syllabus explicitly covers encapsulation, object identity, and polymorphism without inheritance (inheritance is Unit 3). Keep that separation in mind for exam answers.

1. What is Object-Oriented Programming?

Object-Oriented Programming (OOP) is a programming paradigm that organises code around **objects** — self-contained units that bundle together **data (attributes/fields)** and **behaviour (methods)**. The primary goal is to bind data and the functions that operate on it so tightly that no other part of the code can access or modify that data without going through the object's own methods.

OOP models real-world entities directly in code. A **class** is the blueprint; an **object** is a living instance of that blueprint with its own values.

Class vs. Object — the most fundamental distinction

Concept	Meaning	Analogy
Class	A blueprint / template defining attributes and methods	Blueprint of a house
Object	A concrete instance of a class; takes up memory at runtime	Actual built house

```
// A class is a LOGICAL construct – no memory allocated yet
class Student {
    String name;
    int rollNo;
    float marks;

    void display() {
        System.out.println(rollNo + " " + name + " " + marks);
    }
}

// An object is a PHYSICAL entity – memory allocated on the heap
class Main {
    public static void main(String[] args) {
        Student s1 = new Student(); // object created
        s1.name = "Abhilash";
        s1.rollNo = 101;
        s1.marks = 89.5f;
        s1.display();

        Student s2 = new Student(); // another independent object
        s2.name = "Riya";
        s2.rollNo = 102;
        s2.marks = 92.0f;
        s2.display();
    }
}
```

3 Essential Properties of Every Object

- **State** — the current values of an object's attributes. e.g. Student s1 has name="Abhilash", marks=89.5. State can change over time (marks can be updated).
- **Identity** — a unique address/reference that distinguishes one object from all others, even if their states are identical. In Java this is the object's memory address (heap pointer). Two objects with the same state are still two distinct objects.
- **Behaviour** — what the object can do; defined by its methods (display(), deposit(), fly()). Behaviour is shared across all objects of a class but acts on each object's own state.

2. Encapsulation

Encapsulation is the principle of **bundling data (fields) and the methods that operate on that data together inside a single class**, and **restricting direct access** to the internal state from outside the class. It is the OOP mechanism that enforces information hiding.

How it works in Java:

- Declare all fields **private** — they become invisible outside the class.
- Provide **public getter and setter methods** as the only controlled entry points for reading/modifying those fields.
- Setters can enforce **validation rules** (e.g. balance must not go below 0), giving the class full control over its own data.

BankAccount example (canonical encapsulation demo):

```
class BankAccount {
    private double balance;    // private – cannot be touched from outside

    // Constructor: validates initial balance
    public BankAccount(double initialBalance) {
        balance = (initialBalance >= 0) ? initialBalance : 0;
    }

    // Getter – read-only access
    public double getBalance() {
        return balance;
    }

    // Setter with validation – only valid withdrawals allowed
    public void withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
        } else {
            System.out.println("Invalid withdrawal: " + amount);
        }
    }

    public void deposit(double amount) {
        if (amount > 0) balance += amount;
    }
}

class Main {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount(1000);
        System.out.println(acc.getBalance());    // 1000.0
        acc.withdraw(300);
        System.out.println(acc.getBalance());    // 700.0
        acc.withdraw(900);                      // Invalid withdrawal: 900.0
        // acc.balance = -999;                   // ERROR – private field, won't compile
    }
}
```

Student example with full getters/setters:

```
class Student {
    private String name;
    private int    rollNo;
```

```

private float marks;

// Parameterised constructor
public Student(String name, int rollNo, float marks) {
    this.name = name;
    this.rollNo = rollNo;
    setMarks(marks); // use setter so validation runs
}

public String getName() { return name; }
public int getRollNo() { return rollNo; }
public float getMarks() { return marks; }

public void setName(String name) { this.name = name; }
public void setMarks(float marks) {
    if (marks >= 0 && marks <= 100) this.marks = marks;
    else System.out.println("Invalid marks: " + marks);
}

public void display() {
    System.out.println("Roll: " + rollNo + " Name: " + name
        + " Marks: " + marks);
}
}

class Main {
    public static void main(String[] args) {
        Student s = new Student("Abhilash", 101, 89.5f);
        s.display();
        s.setMarks(150); // Invalid marks: 150.0
        s.setMarks(95f);
        s.display();
    }
}

```

Why encapsulation matters (exam-answer points):

Benefit	Explanation
Data hiding	Internal state is hidden; outsiders interact only via the interface
Data integrity	Setters enforce rules, preventing invalid states
Modularity	Class internals can change without breaking code that uses it
Controlled access	Read-only fields (only getter, no setter) can be enforced
Reusability	Well-encapsulated classes plug into any project without side effects

3. Object Identity

Object identity is the property that **every object has a unique existence**, independent of its state. Even if two objects hold exactly the same data, they are distinct entities with distinct identities.

In Java, an object's identity is its **reference (memory address on the heap)**. The `==` operator on two object references checks identity — it returns true only if both references point to the *same* object in memory, not just equal values.

Identity vs. Equality

Concept	Operator / Method	What it checks	Example result
Identity	<code>==</code>	Same object in memory (same reference)	<code>a == b</code> → false (different heap addresses)
Equality (value)	<code>.equals()</code>	Same content / state?	<code>a.equals(b)</code> → true if values match

```
class BankAccount {
    private double balance;
    public BankAccount(double b) { this.balance = b; }
    public double getBalance() { return balance; }
}

class Main {
    public static void main(String[] args) {
        BankAccount a = new BankAccount(1000);
        BankAccount b = new BankAccount(1000); // same state, different object
        BankAccount c = a;                      // c is just another name for a

        System.out.println(a == b); // false - different identities
        System.out.println(a == c); // true  - same object in memory
        System.out.println(a.getBalance() == b.getBalance()); // true - equal values

        // How Java represents identity:
        System.out.println(System.identityHashCode(a)); // e.g. 1829164700
        System.out.println(System.identityHashCode(b)); // e.g. 2018699554 (different!)
        System.out.println(System.identityHashCode(c)); // same as a
    }
}
```

Key exam distinction: object identity is fixed for the lifetime of the object — it never changes even if state changes. Object equality (via `.equals()`) can be overridden to compare by content instead of address. The default `Object.equals()` checks identity, but `String.equals()` checks character-by-character content.

The 'this' keyword and identity

Inside any instance method, `this` is a reference to the **current object's identity** — the specific object the method was called on. It lets you distinguish between instance fields and local/parameter variables with the same name:

```
class Circle {
    private double radius;

    public Circle(double radius) {
        this.radius = radius; // 'this.radius' = field; 'radius' = parameter
    }

    public boolean isSameObject(Circle other) {
        return this == other; // identity check
    }
}
```

```
} }
```

4. Polymorphism

Polymorphism (Greek: *poly* = many, *morphs* = forms) is the ability of a single method name or interface to **behave differently depending on the context**. It makes code flexible, extensible, and easier to maintain.

In Java there are exactly two kinds of polymorphism:

Feature	Compile-Time (Static)	Runtime (Dynamic)
Also called	Static polymorphism	Dynamic polymorphism
Achieved via	Method overloading	Method overriding
Resolved at	Compile time	Runtime (by JVM)
Inheritance needed?	No	Yes (subclass overrides superclass)
Binding type	Early / static binding	Late / dynamic binding
Example trigger	add(int,int) vs add(double,double)	Dog.speak() overriding Animal.speak()

4.1 Compile-Time Polymorphism — Method Overloading

Multiple methods in the **same class** share the same name but differ in their **parameter list** (number, type, or order of parameters). The compiler picks the correct version at compile time based on the arguments you pass.

What CAN differ (determines overloading):

- Number of parameters: add(int a) vs add(int a, int b)
- Type of parameters: add(int a, int b) vs add(double a, double b)
- Order of parameters: show(int a, String s) vs show(String s, int a)

Return type alone does NOT distinguish overloads — the compiler will reject two methods with the same name and parameter types but different return types.

```
class Calculator {
    // Overload 1: two ints
    static int add(int a, int b) {
        return a + b;
    }
    // Overload 2: two doubles
    static double add(double a, double b) {
        return a + b;
    }
    // Overload 3: three ints
    static int add(int a, int b, int c) {
        return a + b + c;
    }
    // Overload 4: String concatenation – same name, different type
    static String add(String a, String b) {
        return a + b;
    }

    public static void main(String[] args) {
        System.out.println(add(2, 4));           // 6           – calls overload 1
        System.out.println(add(5.5, 6.3));       // 11.8          – calls overload 2
        System.out.println(add(1, 2, 3));        // 6           – calls overload 3
    }
}
```

```

        System.out.println(add("Hello", " OOP")); // Hello OOP – calls overload 4
    }
}

```

4.2 Runtime Polymorphism — Method Overriding

A **subclass** provides its own implementation of a method that already exists in its **superclass**, with the **exact same signature** (name + parameter types + return type). At runtime, the JVM looks at the *actual object type* (not the reference type) and dispatches to the correct version — this is called **Dynamic Method Dispatch**.

Rules for overriding:

- Same method name, same parameter list, same (or covariant) return type.
- Access modifier of the overriding method must be the same or **less restrictive** (e.g. protected → public is OK; public → private is NOT).
- Cannot override static, final, or private methods.
- Use `@Override` annotation — the compiler will catch mistakes in the signature.
- `super.methodName()` calls the parent's version from inside the override.

```

class Animal {
    String name;
    Animal(String name) { this.name = name; }

    // Base method – will be overridden
    void speak() {
        System.out.println("Some generic animal sound");
    }
    void breathe() {
        System.out.println(name + " breathes air"); // NOT overridden – inherited as-is
    }
}

class Dog extends Animal {
    Dog(String name) { super(name); }

    @Override
    void speak() {
        System.out.println("Woof! I am " + name);
    }
}

class Cat extends Animal {
    Cat(String name) { super(name); }

    @Override
    void speak() {
        System.out.println("Meow! I am " + name);
    }
}

class Main {
    public static void main(String[] args) {
        // Dynamic method dispatch – reference type Animal, actual type varies
        Animal a;

        a = new Dog("Rex");
    }
}

```

```

a.speak();    // Woof! I am Rex    – Dog's version called at runtime
a.breathe();  // Rex breathes air – inherited, not overridden

a = new Cat("Lily");
a.speak();    // Meow! I am Lily  – Cat's version called at runtime

// Polymorphic array – same code, different behaviour per element
Animal[] zoo = { new Animal("Unknown"), new Dog("Bruno"), new Cat("Simba") };
for (Animal animal : zoo) {
    animal.speak(); // each calls its own version
}
}
}

```

4.3 Overloading vs. Overriding — Key Comparison Table

Point	Method Overloading	Method Overriding
Definition	Same method name, different parameter lists	Same method name and signature in subclass
Class	Same class	Superclass + subclass
Polymorphism type	Compile-time (static)	Runtime (dynamic)
Return type	Can differ (but not to distinguish alone)	Must be same or covariant
Access modifier	Can be anything	Same or less restrictive
static / final methods	Can be overloaded	Cannot be overridden
Binding	Static / early binding	Dynamic / late binding
@Override annotation	Not applicable	Strongly recommended
Inheritance required	No	Yes

4.4 Upcasting and Downcasting

Because of polymorphism, a reference of the parent type can point to an object of the child type. This is what makes the polymorphic array in the example above possible.

- **Upcasting** (implicit, always safe): `Animal a = new Dog("Rex");` — treating a `Dog` as an `Animal`. You can only call methods defined in `Animal` on `a`, but the overridden `Dog` version runs at runtime.
- **Downcasting** (explicit, may throw `ClassCastException`): `Dog d = (Dog) a;` — recasting back to `Dog` to access `Dog`-specific methods. Use `instanceof` before downcasting to be safe.

```
Animal a = new Dog("Rex");    // upcast – implicit and safe
a.speak();                    // Woof! (Dog version, runtime dispatch)

// a.fetch();                  // COMPILER ERROR – fetch() not in Animal

Dog d = (Dog) a;              // downcast – explicit
d.fetch();                    // OK – Dog-specific method now accessible

// Safe pattern:
if (a instanceof Dog) {
    Dog dog = (Dog) a;
    dog.fetch();
}
```

5. Constructors

A **constructor** is a special method called automatically when an object is created (`new ClassName()`). It initialises the object's state. Constructors are key to encapsulation — they let you ensure an object is always born in a valid state.

Type	Signature	When used
Default	No parameters; auto-provided if you write none	Quick test objects with default values
Parameterised	Takes arguments to set initial state	Most common; enforces valid initial state
Copy	Takes an object of the same class	Creating a duplicate with the same values

```
class Rectangle {
    private double width, height;

    // Default constructor
    Rectangle() {
        width = 1.0;
        height = 1.0;
    }

    // Parameterised constructor
    Rectangle(double width, double height) {
        this.width = (width > 0) ? width : 1.0;
        this.height = (height > 0) ? height : 1.0;
    }

    // Copy constructor
    Rectangle(Rectangle other) {
        this.width = other.width;
        this.height = other.height;
    }
}
```

```

// Overloaded method – compile-time polymorphism
double area()          { return width * height; }
double area(double scale) { return width * height * scale; }

public void display() {
    System.out.printf("%.1f x %.1f = %.1f\n", width, height, area());
}
}

class Main {
    public static void main(String[] args) {
        Rectangle r1 = new Rectangle();           // default
        Rectangle r2 = new Rectangle(4.0, 3.0);    // parameterised
        Rectangle r3 = new Rectangle(r2);          // copy

        r1.display();           // 1.0 x 1.0 = 1.0
        r2.display();           // 4.0 x 3.0 = 12.0
        r3.display();           // 4.0 x 3.0 = 12.0
        System.out.println(r2 == r3); // false – same state, different identity
        System.out.println(r2.area(2.5)); // 30.0 – overloaded area()
    }
}

```

6. Access Modifiers

Access modifiers are the mechanism that **enforces encapsulation** at the language level by controlling visibility of classes, fields, and methods.

Modifier	Same class	Same package	Subclass (diff. package)	Everywhere
private	✓	✗	✗	✗
default (no modifier)	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

Rule of thumb for encapsulation: fields → private; methods the world needs → public; helper methods inside the class → private; methods subclasses need to override → protected.

7. Likely Semester-Exam Questions

Q1. Define encapsulation. How is it achieved in Java?

Encapsulation is bundling data and the methods that operate on it inside a class, and restricting direct outside access to the data. Achieved in Java by declaring fields private and providing public getter/setter methods. Setters may include validation logic.

Q2. What is object identity? How does it differ from object equality?

Object identity is the unique existence of an object in memory (its heap address). Two objects are identical only if they are the same object (`==` returns true). Object equality compares content/state, implemented via `.equals()`. Default `Object.equals()` checks identity; it can be overridden to compare values (e.g. `String.equals()`).

Q3. Define polymorphism. What are its two types in Java?

Polymorphism means a single name can behave differently depending on context. (1) Compile-time / static polymorphism — achieved via method overloading (same name, different parameter lists, resolved at compile time). (2) Runtime / dynamic polymorphism — achieved via method overriding (subclass redefines superclass method; correct version chosen by JVM at runtime via dynamic dispatch).

Q4. Differentiate between method overloading and method overriding.

Overloading: same class, different parameter lists, compile-time resolution, no inheritance needed. Overriding: subclass redefines superclass method with same signature, runtime resolution, inheritance required, `@Override` recommended. Return type alone cannot differentiate overloads; overriding must have same or covariant return type.

Q5. What is dynamic method dispatch?

Dynamic method dispatch is the JVM's mechanism of determining which overridden method to call at runtime based on the actual type of the object, not the reference type. It is the underlying engine of runtime polymorphism. Example: `Animal a = new Dog(); a.speak()` calls Dog's `speak()`, not Animal's.

Q6. What is the role of the 'this' keyword in encapsulation?

'this' refers to the current object's identity — the heap address of the object whose method is executing. In setters/constructors it distinguishes instance fields from parameters with the same name: `this.name = name`. It can also be passed as a reference to the current object.

Q7. Can we override a static method in Java?

No. Static methods belong to the class, not to any object, so they are resolved at compile time based on the reference type — this is called method hiding, not overriding. Runtime polymorphism (dynamic dispatch) does not apply to static methods.

Q8. Why is encapsulation important in OOP?

It provides data hiding (internals are hidden), data integrity (validation in setters prevents invalid states), modularity (internal representation can change without breaking clients), controlled access (read-only via getter-only design), and reusability.

End of Unit 2 notes. Sources: GFG (OOP features overview, polymorphism in Java), Code World / arnab7070 (encapsulation, object identity, polymorphism with BankAccount and Animal examples), plus original Java implementations, comparison tables, and exam Q&A written for this syllabus.